

Synonym Root Cause Report

Summary

This audit covers how synonyms/antonyms flow from the raw API through parsing into the app model and finally into UI components. It identifies why unrelated, duplicated, or wrong-meaning synonyms can appear and why navigation can behave inconsistently when tapping a synonym.

Root Causes

- Global aggregation: the app often aggregates synonyms across all meanings/definitions and displays that single list (global view), so synonyms from other senses appear alongside the current sense.
- No normalization at parse time: `parseApiResponse` preserves `m.synonyms` and `d.synonyms` as-is (no trim/lowercase/filter), so casing/whitespace variations and empty strings survive into UI.
- Inconsistent deduping: some places use `Set` to dedupe, but de-duplication is case-sensitive and performed after aggregation rather than during parsing or with normalized keys.
- Meaning-level data not surfaced consistently: `MeaningCard` exposes meaning-level synonyms, but `WordDetailsScreen` and `AudioExperienceScreen` render global related lists instead of wiring per-meaning selections to those handlers.
- Navigation inconsistency: tapping a synonym in `WordDetailsScreen` calls a local `setWord(newWord)` while `AudioExperienceScreen` uses `navigation.setParams(...)`, producing different route/history semantics and potential back-navigation or history misalignment.

Files Involved

- `src/utils/apiParser.ts` — raw API → `DictionaryEntry` mapping (meaning & definition synonyms kept verbatim).
- `src/services/dictionaryService.ts` — HTTP lookup and `parseApiResponse` usage.
- `src/utils/pronunciationHelpers.ts` — `buildRelatedWords` aggregates synonyms across meanings/definitions.
- `src/components/MeaningCard.tsx` — per-meaning synonyms rendering (not consistently used by main screens).
- `src/screens/WordDetailsScreen.tsx` — global synonyms aggregation and `handleSelectRelatedWord` implementation.
- `src/screens/AudioExperienceScreen.tsx` — uses `buildRelatedWords` and `navigation.setParams` when tapping related words.
- `src/services/api.ts` — API base and timeout (context for retries/timeouts).

Evidence / Code Notes

- `parseApiResponse` maps `m.synonyms` and `d.synonyms` directly; see `src/utils/apiParser.ts`.
- Global related-words builder collects from all meanings/definitions: see `buildRelatedWords` in `src/utils/pronunciationHelpers.ts`.

- `WordDetailsScreen` computes `synonyms` via `entry.meanings.flatMap(...)`, producing a global list instead of honoring a selected meaning: see <src/screens/WordDetailsScreen.tsx>.
- In `WordDetailsScreen` tapping a synonym calls `handleSelectRelatedWord` which does `setWord(newWord)` (local state change) rather than updating navigation params: see <src/screens/WordDetailsScreen.tsx>.

Recommended Fixes (priority ordered)

1. Normalize & validate during parsing

- In `parseApiResponse` normalize synonym/antonym strings: trim, remove empty strings, collapse whitespace, and optionally lowercase for deduping keys while retaining display-casing.
- Remove synonyms identical to the entry word.
- Optionally attach provenance to each synonym (`meaningIndex` / `definitionIndex`) so UI can surface meaning-specific synonyms easily.

2. Canonical deduplication

- Deduplicate using a normalized key (e.g., `s.trim().toLowerCase()`) to avoid case/whitespace duplicates.
- Keep a separate display value if original casing matters.

3. Distinguish global vs meaning-level displays

- Use `MeaningCard` for per-meaning synonyms (wire its `onSelectWord` up to a consistent navigation handler).
- Keep the current global related-words UI but label it clearly (e.g., "Related words — all senses").
- If a UI element shows synonyms for the *current* meaning, compute that set from the meaning's provenance metadata instead of global aggregation.

4. Consistent navigation behavior

- Standardize tap handling: always call `navigation.setParams({ word: newWord, phoneticIndex: 0 })` (or navigate to the same `WordDetails` route) so route params, history, and back-button behavior are consistent across screens (`WordDetailsScreen` vs `AudioExperienceScreen`).

5. Defensive UI and caching guards

- Clear or show a loading skeleton immediately on synonym-tap to avoid briefly showing stale synonyms from the previous entry.
- Cap the number of synonyms returned after normalization and sorting by provenance/relevance.

6. Add tests & example fixtures

- Add unit tests for `parseApiResponse` that verify normalization, deduping, and provenance mapping using sample API responses.

Example Words to Reproduce / Demonstrate

- **set** — multiple unrelated senses; aggregating all meanings will surface synonyms that belong to other senses.
- **light** — noun/verb/adjective senses with different synonym sets demonstrating wrong-meaning synonyms.
- **run** — shows duplicates across meanings and varied casing/format.

Suggested Small PR Scope (implementation outline)

1. Update **parseApiResponse** to normalize and annotate synonyms with provenance.
2. Replace ad-hoc **flatMap** aggregations with helpers that accept a provenance filter and a normalization strategy.
3. Update **WordDetailsScreen** to use **MeaningCard** for per-meaning display and to call a single **navigateToWord(newWord)** helper that updates navigation consistently.
4. Add unit tests and a small sample fixture in **/docs/fixtures/** demonstrating problematic API responses.

Next Steps (if you want me to implement)

- I can prepare a minimal patch that normalizes & dedupes in **parseApiResponse**, add provenance metadata, and make **WordDetailsScreen**'s synonym tap behavior consistent. Tell me to proceed and I'll implement the smallest safe change first and add tests.

Report generated by automated code audit.